



Section 4 / 24

Instruction Set Architectures

ISA overall

An **Instruction Set Architecture** (**ISA**) specifies the syntax and semantics of the assembly language on each architecture. It is not just a different syntax but is built in the core design of a processor, as it affects the way and order instructions are executed and their level of complexity. **ISA** mainly consists of the following components:

- Instructions
- Registers
- Memory Addresses
- Data Types

Component	Description	Example
Instructions	The instruction to be processed in the opcode operand_list format. There are usually 1,2, or 3 comma-separated operands.	<code>add rax, 1 , mov rsp, rax , push rax</code>
Registers	Used to store operands, addresses, or instructions temporarily.	<code>rax , rsp , rip</code>

Component	Description	Example
Memory Addresses	The address in which data or instructions are stored. May point to memory or registers.	0xfffffffffaa8a25ff , 0x44d0 , \$rax
Data Types	The type of stored data.	byte , word , double word

These are the main components that distinguish different ISA's and assembly languages. We will cover each of them in more depth in the coming sections, and we'll learn how to use various instructions.

There are two main Instruction Set Architectures that are widely used:

1. Complex Instruction Set Computer (CISC) - Used in Intel and AMD processors in most computers and servers.
2. Reduced Instruction Set Computer (RISC) - Used in ARM and Apple processors, in most smartphones, and some modern laptops.

Let us see the pros and cons of each and the main differences between them.

CISC

The CISC architecture was one of the earliest ISA's ever developed. As its name suggests, the CISC architecture favors more complex instructions to be run at a time to reduce the overall number of instructions. This is done to rely as much as possible on the CPU by combining minor instructions into more complex instructions.

For example, suppose we were to add two registers with the ' `add rax, rbx` ' instruction. In that case, a CISC processor can do this in a single 'Fetch-Decode-Execute-Store' instruction cycle, without having to split it into multiple instructions to fetch `rax` , then fetch `rbx` , then add them, and then store them in `rax`, each of which would take its own 'Fetch-Decode-Execute-Store' instruction cycle.

Two main reasons drove this:

1. To enable `more instructions to be executed at once` by designing the processor to run more advanced instructions in its core.
2. In the past, memory and transistors were limited, so it was preferred to `write shorter programs` by combining multiple instructions into one.

To enable the processors to execute complex instructions, the processor's design becomes more complicated, as it is designed to execute a vast amount of different complex instructions, each of which has its own unit to execute it.

Furthermore, even though it takes a single instruction cycle to execute a single instruction, as the instructions are more complex, each instruction cycle takes more clock cycles. This fact leads to more power consumption and heat to execute each instruction.

RISC

The RISC architecture favors splitting instructions into minor instructions, and so the CPU is designed only to handle simple instructions. This is done to relay the optimization to the software by writing the most optimized assembly code.

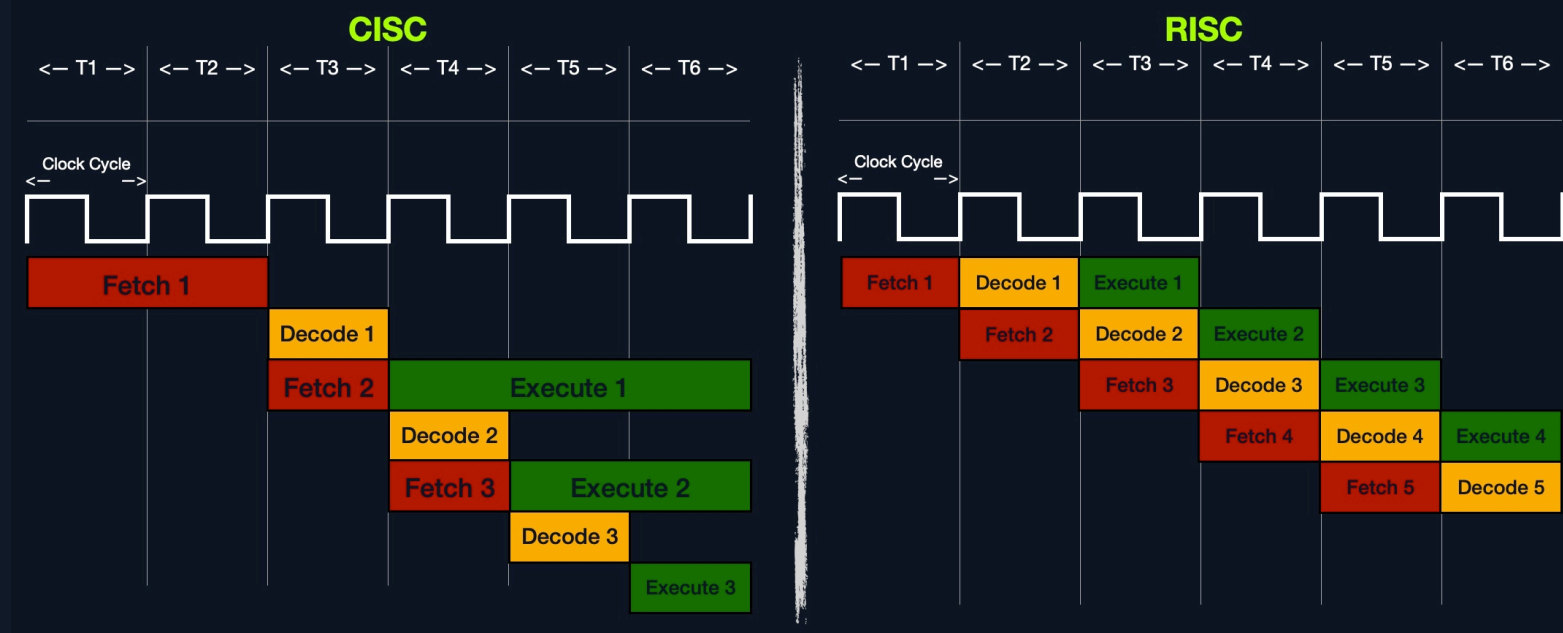
For example, the same previous `add r1, r2, r3` instruction on a RISC processor would fetch `r2`, then fetch `r3`, add them, and finally store them in `r1`. Every instruction of these takes an entire 'Fetch-Decode-Execute-Store' instruction cycle, which leads, as can be expected, to a larger number of total instructions per program, and hence a longer assembly code.

By not supporting various types of complex instructions, RISC processors only support a limited number of instructions (`~200`) compared to CISC processors (`~1500`). So, to execute complex instructions, this has to be done through a combination of minor instructions through Assembly.

It is said that we can build a general-purpose computer with a processor that only supports one instruction! This indicates that we can create very complex instructions using the `sub` instruction only. Can you think of how this may be achieved?

On the other hand, an advantage of splitting complex instructions into minor ones is having all instructions of the same length either 32-bit or 64-bit long. This enables designing the CPU clock speed around the instruction length so that executing each stage in the instruction cycle would always take precisely one machine clock cycle.

The below diagram shows how CISC instructions take a variable amount of clock cycles, while RISC instructions take a fixed amount:



Executing each instruction stage in a single clock cycle and only executing **simple instructions** leads to RISC processors I think It's not quite true. It depends on how we design not only the hardwares, but also the compilers. **consuming a fraction of** the power consumed by CISC processors, which makes these processors ideal for devices that run on batteries, like smartphones and laptops.

CISC vs. RISC

The following table summarizes the main differences between CISC and RISC:

Area	CISC	RISC
Complexity	Favors complex instructions	Favors simple instructions
Length of instructions	Longer instructions - Variable length 'multiples of 8-bits'	Shorter instructions - Fixed length '32-bit/64-bit'

Area	CISC	RISC
Total instructions per program	Fewer total instructions - Shorter code	More total instructions - Longer code
Optimization	Relies on hardware optimization (in CPU)	Relies on software optimization (in Assembly)
Instruction Execution Time	Variable - Multiple clock cycles	Fixed - One clock cycle
Instructions supported by CPU	Many instructions (~1500)	Fewer instructions (~200)
Power Consumption	High	Very low
Examples	Intel, AMD	ARM, Apple

In the past, having a longer assembly code due to a larger number of total instructions per program was a significant disadvantage for RISC processors due to the limited resources in memory and storage. However, today this is no longer as big of an issue, as memory and storage are not as expensive and limited as they used to be in the past.

Furthermore, with new assemblers and compilers writing extremely optimized code on the software level, RISC processors are becoming faster than CISC processors, even in executing and processing heavy applications, all while consuming much less power.

All of this is making RISC processors more common in recent years. RISC may become the dominant architecture in the upcoming years. But as we speak, the overwhelming majority of

computers and servers we will be pentesting are running on Intel/AMD processors with the CISC architecture, making learning CISC assembly our priority. As the basics of all Assembly language variants are pretty similar, learning ARM Assembly should be more straightforward after completing this module.

[< Previous](#)

Section 4 / 24

 +20

[Next](#)[Mark Complete & Next](#)